Supervised Learning Recap

Kianté Brantley
CS 1840 Fall 2026

Announcements

HW3 is out and due 10/29!

Midterm will be in 1 week on Mon 10/26!

Check the course website for Office Hours:
https://harvard-cs-1840.github.io/2026/

Finishing Up From Last Lecture

✓ **1. Simulation lemma:**
What is the performance of π under any estimator $\widehat{P}$

✓ 2. **Algorithm:** estimate $(\widehat{P}, \widehat{r})$ from data
and compute $\widehat{\pi}^\star$—the optimal policy of $(\widehat{P}, \widehat{r})$

3. **Analyzing** the performance $\widehat{\pi}^\star$ under (P, r)

Recap: A Model-based Algorithm

Assume reward r is known (just for analysis simplicity):

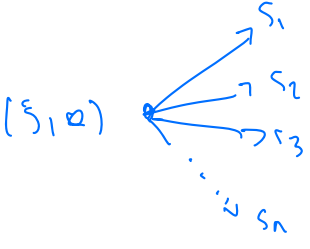
Recap: A Model-based Algorithm

Assume reward r is known (just for analysis simplicity):

1. Model fitting:

$\forall s, a$: collect N next states, $s_i' \sim P(\cdot \mid s, a), i \in [N]$; set

$$\widehat{P}(s' \mid s, a) = \frac{\sum_{i=1}^{N} \mathbf{1}\{s_i' = s'\}}{N};$$



Recap: A Model-based Algorithm

Assume reward r is known (just for analysis simplicity):

1. Model fitting:

$\forall s, a$: collect N next states, $s_i' \sim P(\cdot \,|\, s, a), i \in [N]$; set

$$\widehat{P}(s' \,|\, s, a) = \frac{\sum_{i=1}^{N} \mathbf{1}\{s_i' = s'\}}{N};$$

2. Planning w/ the learned model:

$$\widehat{\pi}^{\star} = \textbf{PI}\left(\widehat{P}, r\right)$$

Recap: estimating mean of Bernoulli distribution

Given: we have a biased coin:

With probability p, it gives +1, and w/ prob 1-p, it gives -1;

Recap: estimating mean of Bernoulli distribution

Given: we have a biased coin:
With probability p, it gives +1, and w/ prob 1-p, it gives -1;

To estimate p: We flip the coin N times independently, get N outcomes, $\{x_i\}_{i=1}^{N}$,

$$x_i \in \{-1, +1\}$$

Recap: estimating mean of Bernoulli distribution

Given: we have a biased coin:

With probability p, it gives +1, and w/ prob 1-p, it gives -1;

To estimate p: We flip the coin N times independently, get N outcomes, $\{x_i\}_{i=1}^N$,

$$x_i \in \{-1, +1\}$$

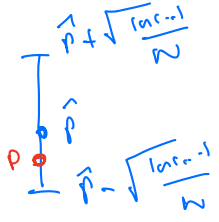
$$\hat{p} = \frac{\sum_{i=1}^N \mathbf{1}\{x_i = +1\}}{N}$$

Recap: estimating mean of Bernoulli distribution

Given: we have a biased coin:

With probability p, it gives +1, and w/ prob 1-p, it gives -1;

To estimate p: We flip the coin N times independently, get N outcomes, $\{x_i\}_{i=1}^N$,



$$x_i \in \{-1, +1\}$$

$$\hat{p} = \frac{\sum_{i=1}^{N} \mathbf{1}\{x_i = +1\}}{N}$$

w/ probability at least, $1 - \delta$ we will have $|\hat{p} - p| \leq O\left(\sqrt{\frac{\ln(1/\delta)}{N}}\right)$

(concentration bound; Hoeffding's inequality; proof out of scope)

Recap: Model-based RL

1. Model fitting:

$\forall s, a$: collect N next states, $s'_i \sim P(\cdot \mid s, a), i \in [N]$; set

$$\widehat{P}(s' \mid s, a) = \frac{\sum_{i=1}^{N} \mathbf{1}\{s'_i = s'\}}{N};$$

Estimating the prob of observing s' at (s, a)

2. Planning w/ the learned model:

$$\widehat{\pi}^\star = \textbf{PI}\left(\widehat{P}, r\right)$$

Recap: Model-based RL

1. Model fitting:

$\forall s, a$: collect N next states, $s_i' \sim P(\cdot \,|\, s, a), i \in [N]$; set

$$\widehat{P}(s' \,|\, s, a) = \frac{\sum_{i=1}^{N} \mathbf{1}\{s_i' = s'\}}{N};$$

Estimating the prob of observing s' at (s, a)

We can show:

$$\|\hat{P}(\cdot \,|\, s, a) - P(\cdot \,|\, s, a)\|_1 \lesssim \sqrt{1/N}, \forall s, a$$

2. Planning w/ the learned model:

$$\widehat{\pi}^\star = \textbf{PI}\left(\widehat{P}, r\right)$$

Finishing Up From Last Lecture

✓ **1. Simulation lemma:**
What is the performance of π under any estimator $\widehat{P}$

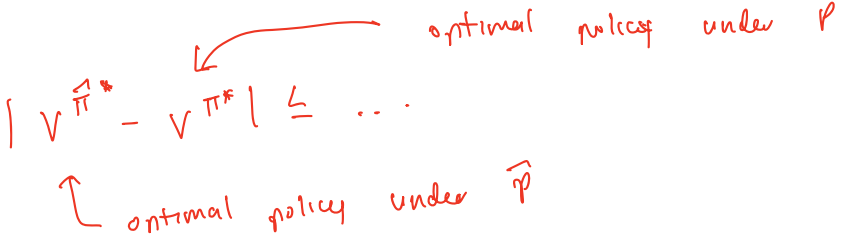
✓ 2. **Algorithm:** estimate $(\widehat{P}, \widehat{r})$ from data
and compute $\widehat{\pi}^\star$—the optimal policy of $(\widehat{P}, \widehat{r})$

3. **Analyzing** the performance $\widehat{\pi}^\star$ under (P, r)

Performance of the learned policy

Now assume that $\hat{P}$ is nearly accurate, i.e., $\|\hat{P}(\cdot\,|s,a) - P(\cdot\,|s,a)\|_1 \le \epsilon, \forall s,a$

How good is $\hat{\pi}^\star = \mathrm{PI}(\hat{P}, r)$?



Performance of the learned policy

Now assume that $\hat{P}$ is nearly accurate, i.e., $\|\hat{P}(\cdot\,|\,s,a) - P(\cdot\,|\,s,a)\|_1 \leq \epsilon, \forall s, a$

How good is $\hat{\pi}^\star = \mathsf{PI}(\hat{P}, r)$?

$V^{\star}(s_0) - V^{\hat{\pi}^\star}(s_0)$

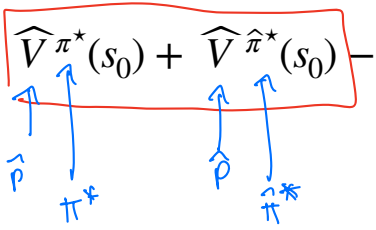
Performance of the learned policy

Now assume that $\hat{P}$ is nearly accurate, i.e., $\|\hat{P}(\cdot\,|s,a) - P(\cdot\,|s,a)\|_1 \le \epsilon, \forall s, a$

How good is $\hat{\pi}^\star = \mathrm{PI}(\hat{P}, r)$?

$$V^\star(s_0) - V^{\hat{\pi}^\star}(s_0)$$

$$\le V^\star(s_0) - \widehat{V}^{\pi^\star}(s_0) + \widehat{V}^{\hat{\pi}^\star}(s_0) - V^{\hat{\pi}^\star}(s_0)$$



Performance of the learned policy

Now assume that $\hat{P}$ is nearly accurate, i.e., $\|\hat{P}(\cdot\,|s,a) - P(\cdot\,|s,a)\|_1 \le \epsilon, \forall s, a$

How good is $\hat{\pi}^\star = \mathrm{PI}(\hat{P}, r)$?

$$V^\star(s_0) - V^{\hat{\pi}^\star}(s_0)$$

Q: why this is true?

$$\le V^\star(s_0) - \widehat{V}^{\pi^\star}(s_0) + \widehat{V}^{\hat{\pi}^\star}(s_0) - V^{\hat{\pi}^\star}(s_0)$$

≥ 0

$\hat{\pi}^*$ is optimal in $\hat{P}$

$\hat{V}^{\hat{\pi}^*} \ge \hat{V}^{\pi^*}$

Performance of the learned policy

Now assume that $\hat{P}$ is nearly accurate, i.e., $\|\hat{P}(\cdot|s,a) - P(\cdot|s,a)\|_1 \leq \epsilon, \forall s, a$

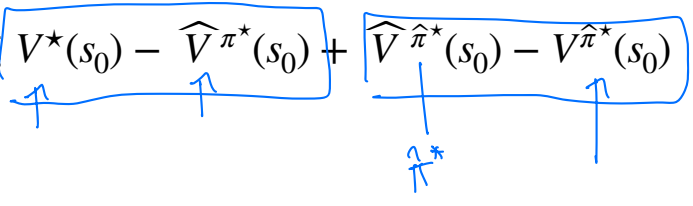
How good is $\hat{\pi}^\star = \text{PI}(\hat{P}, r)$?

$$V^\star(s_0) - V^{\hat{\pi}^\star}(s_0)$$

Q: why this is true?

$$\leq V^\star(s_0) - \widehat{V}^{\pi^\star}(s_0) + \widehat{V}^{\hat{\pi}^\star}(s_0) - V^{\hat{\pi}^\star}(s_0)$$

(Simulation lemma)



Performance of the learned policy

Now assume that $\hat{P}$ is nearly accurate, i.e., $\|\hat{P}(\cdot\,|\,s,a) - P(\cdot\,|\,s,a)\|_1 \le \epsilon, \forall s,a$

How good is $\hat{\pi}^\star = \mathrm{PI}(\hat{P}, r)$?

$$V^\star(s_0) - V^{\hat{\pi}^\star}(s_0)$$

Q: why this is true?

$$\le V^\star(s_0) - \widehat{V}^{\pi^\star}(s_0) + \widehat{V}^{\hat{\pi}^\star}(s_0) - V^{\hat{\pi}^\star}(s_0)$$

(Simulation lemma)

$$\le \frac{\gamma}{(1-\gamma)^2}\left[\mathbb{E}_{s,a\sim d_{s_0}^{\pi^\star}}\|\widehat{P}(\cdot\,|\,s,a) - P(\cdot\,|\,s,a)\|_1 + \mathbb{E}_{s,a\sim d_{s_0}^{\hat{\pi}^\star}}\|\widehat{P}(\cdot\,|\,s,a) - P(\cdot\,|\,s,a)\|_1\right]$$

$d^{\pi^\star}$ $d^{\hat{\pi}^\star}$

Performance of the learned policy

Now assume that $\hat{P}$ is nearly accurate, i.e., $\|\hat{P}(\cdot\,|\,s,a) - P(\cdot\,|\,s,a)\|_1 \le \epsilon, \forall s,a$

How good is $\hat{\pi}^\star = \mathrm{PI}(\hat{P}, r)$?

$$V^\star(s_0) - V^{\hat{\pi}^\star}(s_0)$$

Q: why this is true?

$$\le V^\star(s_0) - \widehat{V}^{\pi^\star}(s_0) + \widehat{V}^{\hat{\pi}^\star}(s_0) - V^{\hat{\pi}^\star}(s_0)$$

(Simulation lemma)

$$\le \frac{\gamma}{(1-\gamma)^2}\left[\mathbb{E}_{s,a\sim d_{s_0}^{\pi^\star}}\|\widehat{P}(\cdot\,|\,s,a) - P(\cdot\,|\,s,a)\|_1 + \mathbb{E}_{s,a\sim d_{s_0}^{\hat{\pi}^\star}}\|\widehat{P}(\cdot\,|\,s,a) - P(\cdot\,|\,s,a)\|_1\right]$$

$$\le \frac{2\gamma}{(1-\gamma)^2}\cdot\epsilon,$$

$\epsilon = \sqrt{\frac{S\ln(2)A/\delta)}{N}}$

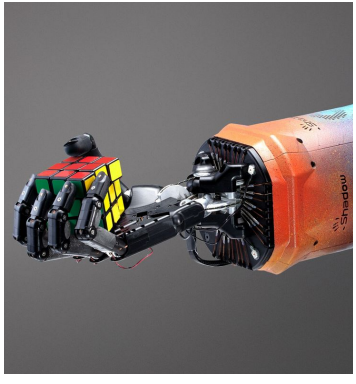
Recap:

So far, three learning algorithms:
TD learning, Q learning, and model-based RL

Limitation: they only work for small MDPs with discrete states and actions

Recap:

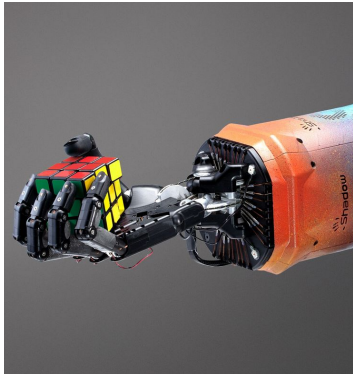
Real world problems often have continuous state or extremely large number of states





Recap:

Real world problems often have continuous state or extremely large number of states





Cannot hope to **enumerate** all possible state-actions in reality…

Starting from today:

Making RL work for large-scale MDPs with the help from supervised learning
(e.g, Deep Learning)

Outline

Recap supervised learning

Tutorial on PyTorch and Gym

Multi-class classification

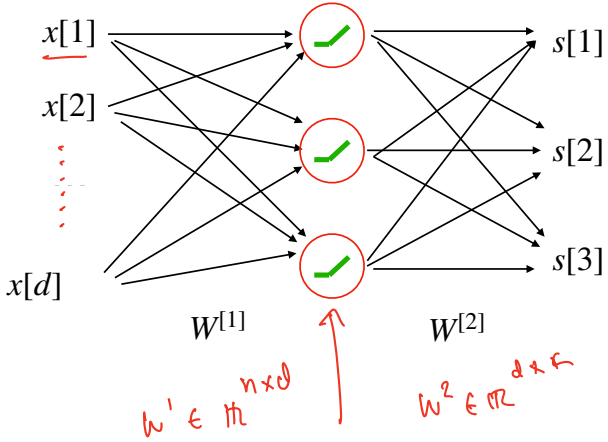
Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \{1,2,\ldots,K\}$

Goal: learn the distribution over labels $P(\cdot \mid x)$

Multi-class classification

Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \{1,2,\ldots,K\}$

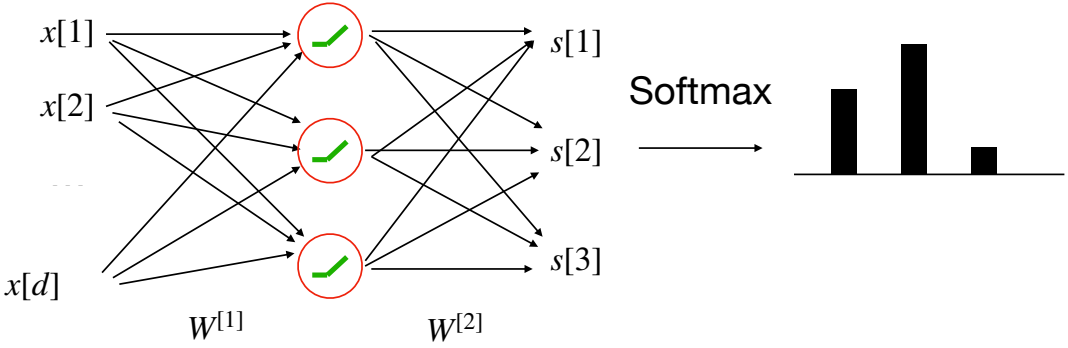
Goal: learn the distribution over labels $P(\cdot \mid x)$



Multi-class classification

Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \{1,2,\ldots,K\}$

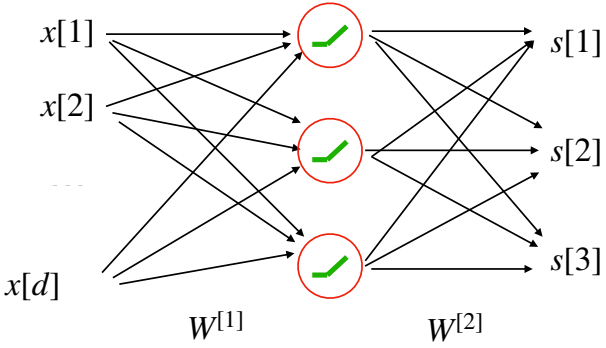
Goal: learn the distribution over labels $P(\cdot \mid x)$



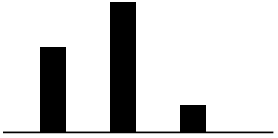
Multi-class classification

Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \{1,2,\ldots,K\}$

Goal: learn the distribution over labels $P(\cdot \mid x)$



Softmax

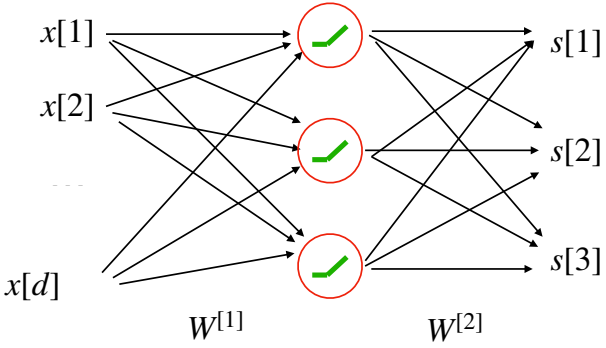


$$P(y = i \mid x) = \frac{\exp(s[i])}{\sum_{j=1}^{K} \exp(s[j])}$$

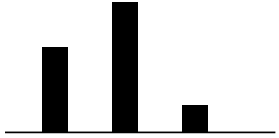
Multi-class classification

Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \{1,2,\ldots,K\}$

Goal: learn the distribution over labels $P(\cdot \mid x)$



Softmax



$$P(y = i \mid x) = \frac{\exp(s[i])}{\sum_{j=1}^{K} \exp(s[j])}$$

$$P_\theta(\cdot \mid x) = \text{softmax}\left(W^{[2]}\text{ReLu}(W^{[1]}x)\right)$$

Multi-class classification

Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \{1,2,\ldots,K\}$

Goal: learn the distribution over labels $P(\cdot \mid x)$

Loss function:

Negative log-likelihood

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

Multi-class classification

Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \{1,2,\ldots,K\}$

Goal: learn the distribution over labels $P(\cdot \mid x)$

Loss function:

Negative log-likelihood

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

$$\hat{\theta} = \arg\min_\theta \ell(\theta)$$

Maximize the likelihood of labels given features

Regression

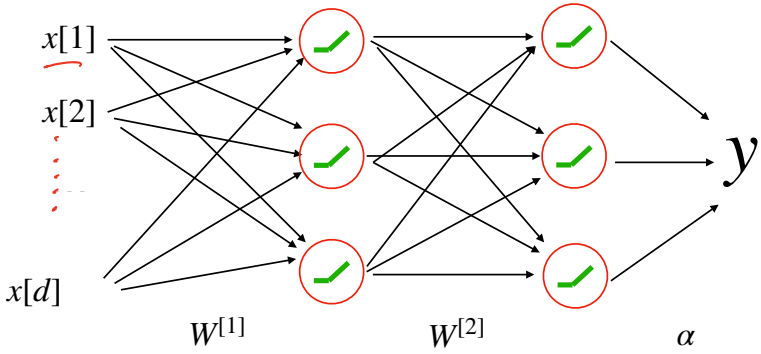
Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \mathbb{R}, x \sim p, y \sim p(\,.\,|\,x)$

Goal: learn the **Bayes optimal** $\mathbb{E}[y\,|\,x]$

Regression

Input: $\mathscr{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \mathbb{R}, x \sim p, y \sim p(\,.\,|\,x)$

Goal: learn the **Bayes optimal** $\mathbb{E}[y\,|\,x]$



$$y = \alpha^{\top}\text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right)$$

Regression

Input: $\mathcal{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \mathbb{R}$

Goal: given x, learn the **Bayes optimal** $\mathbb{E}[y \mid x]$

Loss function:

Mean square error (MSE)

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(x) - y\right)^2$$

Regression

Input: $\mathcal{D} = \{x, y\}, x \in \mathbb{R}^d, y \in \mathbb{R}$

Goal: given x, learn the **Bayes optimal** $\mathbb{E}[y \mid x]$

Loss function:

Mean square error (MSE)

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(f_\theta(x) - y\right)^2$$

$$\hat{\theta} = \arg\min_\theta \ell(\theta)$$

Minimize the mean squared error

What we can hope from supervised learning?

We expect the learned regressor/classifier do well **under the same distribution** where training data is sampled

What we can hope from supervised learning?

We expect the learned regressor/classifier do well **under the same distribution** where training data is sampled

e.g., for regression, under cerntain assumptions

$$\mathbb{E}_{x\sim p}\left(f_{\hat{\theta}}(x) - \mathbb{E}[y\,|\,x]\right)^2 \to 0, \text{ as } N \to \infty$$

What we can hope from supervised learning?

We expect the learned regressor/classifier do well **under the same distribution** where training data is sampled

e.g., for regression, under cerntain assumptions

$$\mathbb{E}_{x\sim p}\left(f_{\hat{\theta}}(x) - \mathbb{E}[y\,|\,x]\right)^2 \to 0, \text{ as } N \to \infty$$

Dist where training data is sampled

Generalization

Supervised learning exhibits generalization ability, as long as test samples are sampled from the same training dist

e.g., Classifer trained on large-scale cat / dog images can classifier **unseen** cat or dog images

Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} - \ln P_\theta(y^i \mid x^i)$$

Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N}\sum_{i=1}^{N} -\ln P_\theta(y^i \,|\, x^i)$$

GD: $\theta^{t+1} = \theta^t - \eta \nabla_\theta \ell(\theta^t)$

Let $\theta^{t+1} = \theta^t + \Delta\theta$

first order taylor approximation

$\ell(\theta^{t+1}) \approx \ell(\theta^t) + \nabla_\theta \ell(\theta^t)^\top [\theta - \theta^t]$

$\ell(\theta^t + \Delta\theta) \approx \ell(\theta^t) + \nabla_\theta \ell(\theta^t)^\top (\Delta\theta)$

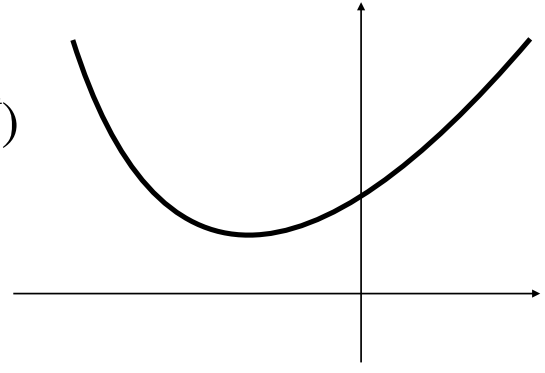
$\ell(\theta^t + \Delta\theta) - \ell(\theta^t) \approx \nabla_\theta \ell(\theta^t)^\top (\Delta\theta)$

Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

GD: $\theta^{t+1} = \theta^t - \eta \nabla_\theta \ell(\theta^t)$



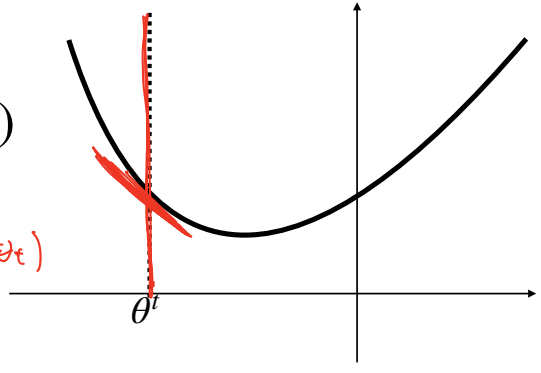
Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

GD: $\theta^{t+1} = \theta^t - \eta \nabla_\theta \ell(\theta^t)$

$\ell(\theta) \approx \ell(\theta^t) + \nabla \ell(\theta^t)^\top (\theta - \theta_t)$

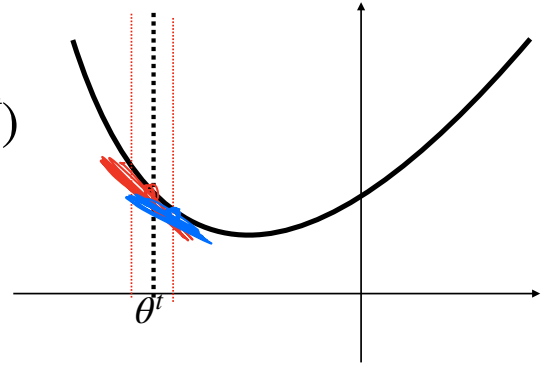


Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

GD: $\theta^{t+1} = \theta^t - \eta \nabla_\theta \ell(\theta^t)$

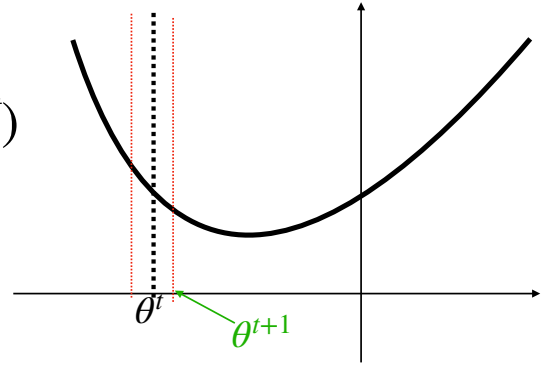


Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

GD: $\theta^{t+1} = \theta^t - \eta \nabla_\theta \ell(\theta^t)$



Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

$$\theta^{t+1} = \theta^t - \eta \widetilde{\nabla_\theta \ell}(\theta^t)$$

Optimization

We focus on first-order optimization techiqune: (stochatsic) gradient descent

$$\ell(\theta) = \frac{1}{N} \sum_{i=1}^{N} -\ln P_\theta(y^i \mid x^i)$$

$$\theta^{t+1} = \theta^t - \eta \widetilde{\nabla_\theta \ell}(\theta^t)$$

Q: how to get this unbiased estimate of the gradient?

Optimization

Often we use adaptive gradient methods such as Adagrad or Adam:

In high level, adaptively set learning rates for different coordinates and time

Outline

Recap supervised learning

Tutorial on PyTorch and Gym

Outline



Recap supervised learning

Tutorial on PyTorch and Gym

Feedback and Attendance

https://bit.ly/4gYn2nC

